



2.6. Коллекции: сеты и мапы

Привет!

На связи шпаргалка урока 2.6. Коллекции: сеты и мапы. В этом материале мы разберем устройство коллекций изнутри, а именно: каким образом в них хранятся элементы и как происходит добавление/удаление/получение и поиск элемента.

Время прочтения: 15 минут.

Понимание этих механизмов поможет вам оптимально выбирать структуру, а также успешно проходить собеседования на младшего разработчика в различных компаниях, так как вопросы о работе коллекций одни из самых популярных.

Следует учесть, что код в стандартной библиотеке мог писаться несколько десятков лет назад, и лучше не брать из него каких-то новых практик вроде присваивания значений в скобках условия if и тому подобное. Такие решения могут считаться плохой практикой в современной разработке.

Ассоциативный массив (map)

Базовые операции

Проверка на наличие элемента

Итерация по ассоциативному массиву

Списки

Ассоциативный массив

HashMap

Как хранятся элементы

Как добавляются элементы

Как удаляются элементы

Как происходит поиск и получение элемента

Множества

HashSet

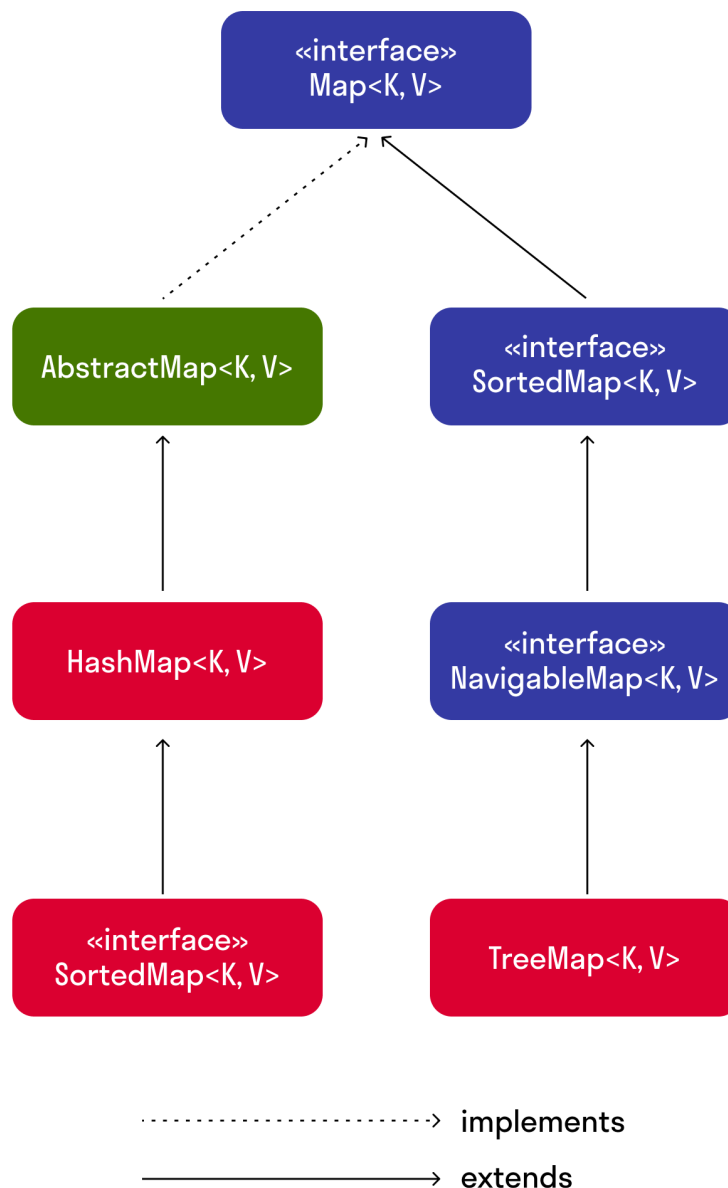
Как хранятся элементы

Как работает HashSet?

Ассоциативный массив (map)

Вот мы и переходим к самой сложной с точки зрения реализации, но прекрасной с точки зрения функционала коллекции, которая коллекцией (с точки зрения реализации интерфейсов) не является.

Ассоциативный массив или map — это хранилище по принципу ключ-значение, где ключи уникальны, а значения могут повторяться.



Базовая реализация называется `HashMap`.

Как видно из названия, построена она на том же `hashCode`, что и `HashSet`.

Это значит, что ключами должны быть неизменяемые объекты, ведь при изменении состояния объекта (значений в его полях и тому подобное) меняется и `hashCode`, а значит, мы теряем возможность быстро найти этот элемент в коллекции, ведь он был добавлен в хранилище по одному хеш-коду, а после изменения состояния объекта поиск будет идти по другому хеш-коду.

В связи с этим обычно в качестве ключей используются строки.

Создается мапа с помощью следующего вида строки:

```
Map<String, Integer> map = new HashMap<>();
```

Первым в < > указывается тип ключа, вторым — тип значения.

Базовые операции

Базовыми операциями мапы являются добавление, получение и удаление элемента.

Рассмотрим эти операции.

Представим, что у нас есть список контактов, где в качестве ключа выступает фамилия нашего контакта (или любое другое слово), а значением является номер. Так как номера любят указывать с плюсом (+), минусом (–), круглыми скобками(и), номер тоже будет строкой.

Создадим мапу Map<String, String>.

```
Map<String, String> phoneBook = new HashMap<>();
```

Элементы в мапу добавляются с помощью метода put.

В этот метод мы должны передать два параметра, а именно: ключ и значение.

Добавим в нашу мапу первые два контакта.

```
phoneBook.put("Ivan", "123-12-12");  
phoneBook.put("Dmitriy", "345-34-34");
```

Теперь в нашей мапе два контакта.

Попробуем добавить еще одного Ивана.

```
phoneBook.put("Ivan", "122-12-12");
```

Количество контактов в нашей мапе не изменится, так как новое добавление по старому ключу перезапишет старое значение и значении по ключу "Ivan" будет лежать тот номер, что был добавлен по этому ключу последним.

Попробуем получить нашего Ивана.

Это происходит с помощью метода `get`, в качестве параметра которому указывается ключ.

```
System.out.println(phoneBook.get("Ivan"));
```

После выполнения этой строки будет напечатано 122-12-12, то есть номер, что был добавлен Ивану последним.

Теперь удалим Дмитрия.

Это осуществляется методом `remove`, который в качестве параметра получает ключ.

```
phoneBook.remove("Dmitriy");
```

После этого попробуем получить Дмитрия из карты.

```
System.out.println(phoneBook.get("Dmitriy"));
```

Будет напечатано `null`, так как никакого Дмитрия в карте больше нет.

Проверка на наличие элемента

Чтобы не натолкнуться на `null` в случае получения элемента, а затем не выхватить `NullPointerException` в момент вызова метода у этого самого полученного `null`, нам следует проверить, есть ли такой элемент в нашей карте.

Для проверки, что элемент есть в карте, используется метод `containsKey`.

Перепишем получение Дмитрия с учетом того, что может вернуться `null`.

```
if (phoneBook.containsKey("Dmitriy")) {  
    System.out.println(phoneBook.get("Dmitriy"));  
} else {  
    System.out.println("Контакт не найден");  
}
```

Карта является достаточно популярной структурой, потому многие ситуации уже проработаны в ней.

Например, если нам вдруг необходимо проверить наличие значения, в карте реализован метод `containsValue`.

Учтите, что поиск по значению идет линейно, то есть перебором всех значений, потому использовать этот метод следует в крайнем случае.

```
if (phoneBook.containsValue("122-12-12")) {  
    System.out.println("Номер 122-12-12 уже присутствует в контакт-листе");  
}
```

К сожалению, получить ключ по значению нельзя, в связи с чем метод `containsValue` позволяет только убедиться в наличии значения в мапе.

Итерация по ассоциативному массиву

Так как мапа не является реализацией интерфейса `Iterable`, итерироваться напрямую по мапе нельзя.

В связи с чем в мапе были реализованы методы, собирающие из определенной части мапы множество, а уже по этому множеству можно итерироваться.

Вернемся к нашему `phoneBook`.

Чтобы проитерироваться по парам «ключ-значение», нам нужно написать следующий цикл:

```
for (Map.Entry<String, String> contact: phoneBook.entrySet()) {  
    System.out.println("Контакт " + pair.getKey() + ": " + pair.getValue());  
}
```

Этот цикл выведет все пары «ключ-значение» в консоль.

Если нам нужны только ключи или только значения, вместо `entrySet` используются методы `keySet` или `values`.

```
for (String name: phoneBook.keySet()) {  
    System.out.println("В мапе присутствует контакт по имени " + name);  
}  
  
for (String phoneNumber: phoneBook.values()) {  
    System.out.println("В мапе присутствует номер телефона " + phoneNumber);  
}
```

▼ Дополнительно. Двухуровневые методы

В Java 8 работать с `map` стало проще и приятнее.

Были добавлены несколько методов, которые могут сильно вам облегчить жизнь.

`getOrDefault` — метод принимает два параметра: ключ и дефолтное значение, если элемент по данному ключу отсутствует.

Этот метод избавляет вас от необходимости проверять наличие элемента в мапе, возвращая в случае отсутствия какое-то значение «по умолчанию».

`remove` — метод принимает два параметра, а именно: ключ и значение. Если в мапе имеется элемент, который совпадает и по ключу, и по значению, он будет удален.

`computeIfAbsent/Present` — эти два метода позволяют добавить или заменить значение в случае, когда оно отсутствует/присутствует.

`putIfAbsent` — метод позволяет добавить элемент в мапу только в том случае, если этот элемент (ключ) отсутствует в ней.

`merge` — метод позволяет добавить элемент, если он отсутствует, или с помощью каких-то манипуляций добавить новый элемент к текущему. Например, если мапа используется в качестве счетчика каких-то предметов, с помощью метода `merge` можно или добавить значение 1 (если ранее предмет не встречался), или увеличить текущее значение (если ранее предмет встречался) на 1.

Использование этих методов возможно только в том случае, если вы уже освоили лямбда-выражения, так как эти методы в качестве параметров принимают именно их.

Списки

Устройство двух видов списков, что были упомянуты ранее, отличается максимально. Общего у них только то, что оба являются списками, т. е. упорядоченными структурами, содержащими любое количество элементов без привязки к уникальности.

Ассоциативный массив

Переходим к самой сложной с точки зрения реализации структуре данных. В ней совмещены реализации двух листов, а еще присутствует самобалансирующееся дерево.

Дерево позволяет искать элементы быстрее, так как располагает их в порядке увеличения/уменьшения хеш-кода. И вы проверите значительно меньше элементов для нахождения нужного.

Эту структуру мы изучим в дальнейшем, пока познакомимся с общими идеями.

HashMap

Как хранятся элементы

- ▼ Как ни странно, хранятся элементы в массиве, но не в обычном.

```
The table, initialized on first use, and resized as necessary. When allocated, length is always a power of two. (We also tolerate length zero in some operations to allow bootstrapping mechanics that are currently not needed.)  
  
transient Node<K,V>[] table;
```

- ▼ Используется уже знакомый нам ранее класс `Node`, но для нашей мапы он работает иначе.

```
Basic hash bin node, used for most entries. (See below for TreeNode subclass, and in LinkedHashMap for its Entry subclass.)  
  
static class Node<K,V> implements Map.Entry<K,V> {  
    final int hash;  
    final K key;  
    V value;  
    Node<K,V> next;  
  
    @Contract(pure = true)  
    Node(int hash, K key, V value, Node<K,V> next) {  
        this.hash = hash;  
        this.key = key;  
        this.value = value;  
        this.next = next;  
    }  
}
```

Как видите, в данной реализации узла есть хеш (тот самый хеш-код из метода `hashCode` вашего объекта), ключ и

значение, а также ссылка на следующий элемент.

Именно в таких узлах хранятся объекты в мапе.

Следует знать, что у массива `table` есть `loadFactor`, при превышении которого массив расширяется. Элементы в зависимости от реализации остаются в тех же ячейках или переносятся в заранее известные места.

Как добавляются элементы

▼ В тот момент, когда мы вызываем метод `put`, происходит следующее.

```
Associates the specified value with the specified key in this map. If the map previously contained a mapping for the key, the old value is replaced.  
Params: key – key with which the specified value is to be associated  
        value – value to be associated with the specified key  
Returns: the previous value associated with key, or null if there was no mapping for key. (A null return can also indicate that the map previously associated null with key.)  
  
public V put(K key, V value) {  
    return putVal( hash: hash( key: key), key: key, value: value, onlyIfAbsent: false, evict: true);  
}
```

Последние два параметра можно опустить, они не сильно влияют на сам процесс. Здесь, как мы видим, происходит вычисление хеша на основе ключа (результата его метода `hashCode`), а затем вызов метода `putVal`.

Реализация метода `putVal` очень сложная в плане читабельности, потому скриншот прилагать не стоит.

Если вкратце, то его алгоритм работы такой:

1. На основе полученного хеша и размера массива `table` рассчитать номер ячейки для хранения объекта ключ-значение.
2. Создать объект `Node`, обернув в него наш объект.
3. Если в ячейке уже лежит какой-то объект, то проверить, что новый объект и существующие объекты не равны по ключам.
4. Если равны, то затереть старый объект, заменив его новым.
5. Если не равны, то взять старый объект, положить ссылку на него в новый объект (поле `next`), а новый объект уже положить в эту ячейку. В результате

в ячейке получится некий аналог связанного списка.

6. В тот момент, когда в одной ячейке накапливается 7 и более элементов, превратить образованный в ячейке связанный список из Node в дерево.
7. В тот момент, когда элементов в ячейке становится 6 или менее, конвертировать дерево обратно в связанный список.

Как вы уже могли понять, изменение объекта-ключа приведет к изменению хеш-кода.

Следовательно, при добавлении или поиске элемента по этому ключу хеш-код может отличаться.

В такой ситуации поиск или добавление могут быть сломаны.

Поэтому рекомендуется использовать в качестве ключа неизменяемые объекты или, если это невозможно, формировать в методе `hashCode` результат по бизнес-ключам, которые не должны изменяться (например, `id`).

Как удаляются элементы

Удаляются элементы так же, как и добавляются.

Разберем действия по шагам:

1. При получении в метод `remove` ключа происходит вычисление его хеша.
2. На основе хеша и размерности вычисляется ячейка.
3. Если в этой ячейке один элемент и ключи совпадают, он удаляется.
4. Если в этой ячейке несколько элементов, происходит поиск через сравнение ключей.
5. Если ключи совпадают, объект удаляется, а поля `next` остальных узлов переписываются, исключая объект из цепи.
6. Если после сравнения всех элементов совпадающий ключ не найден, удаления не происходит.

Как происходит поиск и получение элемента

Как вы уже могли понять по двум разделам выше, всё строится на поиске ячейки на основе хеш-кода ключа + размерности массива `table`.

Методы `get` и `containsKey` осуществляют тот же поиск по хешу + размерности, что и удаление/добавление.

После нахождения ячейки ключи сравниваются и элемент или возвращается, или возвращается `true/false` в качестве маркера наличия элемента.

`containsValue` проходит линейно по всем ячейкам массива `table`, проверяя все узлы на предмет равенства поля `value` параметру метода.

Множества

HashSet

Как хранятся элементы

Чтобы понять принцип работы `HashSet`, нужно понять принцип работы `HashMap`. Его мы обсудили выше.

Почему они связаны?

- ▼ Чтобы разобраться, почему эти две структуры связаны, достаточно взглянуть на эти строки кода.

```
private transient HashMap<E, Object> map;

// Dummy value to associate with an Object in the backing Map
private static final Object PRESENT = new Object();

Constructs a new, empty set; the backing HashMap instance has default initial
factor (0.75).

@Contract(pure = true)
public HashSet() { map = new HashMap<>(); }
```

Как видите, всю работу за `HashSet` выполняет внутреннее поле `map`.

Как работает HashSet?

В качестве ключа подаются наши с вами объекты, а в качестве значения — объект `PRESENT`. Все значения в этой самой мапе являются ссылками на один и тот же объект-заглушку `PRESENT`.

- ▼ В остальном процесс абсолютно идентичен тому, что происходит в мапе, ведь методы выглядят так.

```

Returns the number of elements in this set (its cardinality).
Returns: the number of elements in this set (its cardinality)
public int size() { return map.size(); }

Returns true if this set contains no elements.
Returns: true if this set contains no elements

@Contract(pure = true)
public boolean isEmpty() { return map.isEmpty(); }

Returns true if this set contains the specified element. More formally, returns true if and only if this
set contains an element e such that Objects.equals(o, e).
Params: o – element whose presence in this set is to be tested
Returns: true if this set contains the specified element

@Contract(pure = true)
public boolean contains(Object o) { return map.containsKey(key: o); }

Adds the specified element to this set if it is not already present. More formally, adds the specified
element e to this set if this set contains no element e2 such that Objects.equals(e, e2). If this
set already contains the element, the call leaves the set unchanged and returns false.
Params: e – element to be added to this set
Returns: true if this set did not already contain the specified element

public boolean add(E e) { return map.put(e, PRESENT)!=null; }

Removes the specified element from this set if it is present. More formally, removes an element e
such that Objects.equals(o, e), if this set contains such an element. Returns true if this set
contained the element (or equivalently, if this set changed as a result of the call). (This set will not
contain the element once the call returns.)
Params: o – object to be removed from this set, if present
Returns: true if the set contained the specified element

public boolean remove(Object o) { return map.remove(key: o)==PRESENT; }

Removes all of the elements from this set. The set will be empty after this call returns.
public void clear() { map.clear(); }

```

Как видите, всю работу за HashSet выполняет HashMap, которой делегируется выполнение всех операций с объектами, хранимыми в сете.

Разработчиков просто избавили от необходимости создавать мапу в тех ситуациях, когда значения им не требуются.